Code That Moves Metal

Programming 101 for FIRST Tech Challenge

Java · Objects · State machines · FTC SDK

Version 2 | Built from: Learn Java for FTC (Alan G. Smith) · Game Manual 0

Pedro Pathing is covered in a separate deck.

HOW TO USE THIS DECK

Five stops. Then you can read robot code.

0	The loop — how a robot actually thinks
1	Java — the words you type
2	Object oriented concepts — how code gets organised
3	Thinking first — flowcharts, events, state machines
4	The FTC SDK — reaching the real motors

You do not have to memorise this.
You have to **recognise the words**. Keep the PDF — it is a lookup sheet for the whole season.

Every slide has a **Read more** line at the bottom pointing to the real book, guide or docs page it came from.

Each part opens with the words you will meet, and closes with the same list as a recap. If you can explain both lists, you have got it.

Read more: Learn Java for FTC (free PDF) · gm0.org

Your robot is a goldfish with excellent reflexes.

It does not remember “the plan”. About fifty times every second it wakes up, looks around, decides one small thing, and moves. Then it forgets and does it again.



> Like riding a bicycle. You do not steer once and let go. You make tiny corrections, constantly, without thinking about it.

Here is a real program. You cannot read it yet.

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.OpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;

@TeleOp()
public class HelloWorld extends OpMode {
    @Override
    public void init() {
        telemetry.addData("Hello", "World");
    }

    @Override
    public void loop() {
    }
}
```

Sixteen lines. Right now most of it looks like noise.

> **By the end of this deck you will be able to explain every single line of it.** We come back to this exact program near the end.

That is the whole point of the next hour.

Read more: Book §1.2.4 “Hello, World”, Listing 1.1

1

Java

The words you type. Java is a very fussy way of writing instructions down so a machine can follow them exactly.

IN THIS PART

1 One class, one file
2 Variables and types
3 Scope
4 if / else
5 while and for
6 switch
7 enum
8 Methods
9 public and private
10 static and final
11 Annotations

Sixteen words you are about to meet.

`int`	whole number
`double`	number with decimals
`boolean`	true or false
`String`	text
`if / else`	make a decision
`while / for`	repeat something
`switch`	pick one of many options
`enum`	a short list of allowed names

`return`	hand an answer back
`void`	hands nothing back
`public`	everyone may use it
`private`	only this class may use it
`static`	one copy shared by all
`final`	set once, never changes
`@TeleOp`	show up on the Driver Station
`@Override`	deliberately replacing a parent method

Read more: Book Chapters 2, 4, 5, 12, 19

One class, one file. Same name, every time.

File on disk	Class inside it
`HelloWorld.java`	`public class HelloWorld`
`Shooter.java`	`public class Shooter`
`Intake.java`	`public class Intake`

Spelling and capital letters must match **exactly**. `shooter.java` holding `class Shooter` will not build.

- In Java, **all code lives inside a class**. There is no code floating loose.
- A `package` is just the folder the file sits in. The `package` line at the top says which one.
- Android Studio writes that line for you when you create the file.

One exception you will meet soon: a small `enum` is often written **inside** another class rather than given its own file. That is normal and allowed.

Read more: Book §1.2.1 “Some terminology”, §5.4 “Creating your own classes”

A variable is a box with a name taped on the front.

```
int teamNumber;
double motorSpeed;
boolean touchSensorPressed;

teamNumber = 16072;
motorSpeed = 0.5;
```

Or do both at once:

```
double motorSpeed = 0.5;
```

- You must say the **type**, then the **name**, then a `;`
- Names use camelCase: `motorSpeed`, `gyroHeading`
- You can put a new value in the box any time. The newest one wins.

> If you never put anything in the box, numbers start at **0** and booleans start at **false**. Java does not leave boxes empty.

Read more: Book Chapter 2 “Variables and Data Types”

Java has eight basic types. You will use three.

Type	Holds	Examples	When you use it
`int`	whole numbers, no decimal point	`5` `-3` `16072`	counting things, team number
`double`	numbers with a decimal point	`0.5` `-1.0` `3.14`	motor power, stick position, angles
`boolean`	only `true` or `false`	`true` `false`	is the button pressed? is the gate open?

And one more: `String`, for text. `String myName = "Alan Smith";` — note the capital S. That is because `String` is secretly a **class**, not a basic type. More on classes in Part 2.

Read more: Book §2.1 “Primitive Data Types”, §2.2 “String”

Curly braces are walls. Variables cannot climb them.

```
public void loop() {
    int x = 5;
    // x is visible here
    {
        int y = 4;
        // x and y are visible here
    }
    // only x is visible here
}
```

A variable lives from the line where you create it until the `}` that closes its block.

Why this bites you: if you make a variable inside `loop()`, it is **brand new every single time** the loop runs. It cannot remember anything. To remember, it has to live up at the class level. This turns out to matter enormously — see Part 3.

Read more: Book §2.3 “Scope”

`if` asks a question. `else` is the other door.

```
if (motorOn) {
    board.setMotorSpeed(0.5);
} else {
    board.setMotorSpeed(0.0);
}
```

```
if (gamepad1.a && !aAlreadyPressed) {
    motorOn = !motorOn;
}
```

Symbol	Means	Reads as
`&&`	AND	both must be true
`||`	OR	either one will do
`!`	NOT	flip it around
`==`	is equal to	note: two equals signs
`=`	put this value in	one equals sign

The classic beginner bug: writing `=` when you meant `==`.

Read more: Book Chapter 4 “Making decisions”, Listing 12.1

`while` repeats. `for` counts. Use both sparingly.

```java
// keep going until something changes
while (angle > 180) {
    angle -= 360;
}

// do this exactly four times
for (int i = 0; i < 4; i++) {
    telemetry.addData("Motor", i);
}
```

Read this twice.
In FTC you almost never need a long loop, because `loop()` is **already** repeating for you fifty times a second.

A loop that waits for something to happen will freeze your whole robot while it waits. Nothing else can run.

Short loops that finish instantly — like the two on the left — are completely fine.

Read more: Book §4.5 “For”, Listing 5.3

`switch` picks one option out of many.

```
switch (level) {
    case 1:
        setArmHeight(100);
        break;
    case 2:
        setArmHeight(250);
        break;
    case 3:
        setArmHeight(400);
        break;
    default:
        setArmHeight(0);
}
```

A tidy way of writing a long chain of `if / else if / else if`. Same result, far easier to read.

`switch (x)`	look at this one value
`case 1:`	if it is 1, do this
`break;`	stop here, do not fall into the next case
`default:`	none of the above matched

Forget the `break;` and Java carries straight on into the next case. It is legal, so the compiler will not warn you.

Read more: Book §12.2.1 “Using the switch statement”, Listing 12.3

An enum is a short list of allowed names.

```
enum Alliance {
    RED,
    BLUE
}

enum GateState {
    OPEN,
    CLOSED,
    MOVING
}

GateState gate = GateState.CLOSED;
```

Once a variable is an `enum`, **nothing outside the list can ever be put in it.** The compiler simply refuses.

Instead of	Problem
`int gate = 2;`	what was 2 again?
`String gate = "open";`	`"Open"` and `"opne"` both compile, then fail
`GateState.OPEN`	readable **and** typo-proof

> Names are written in CAPITALS by convention. A small enum usually lives inside the class that uses it.

Read more: Book §12.2.3 “Enumerated types”, Listing 12.5

A method is a machine: things go in, a thing comes out.

```
double squareInputWithSign(double input) {
    double output = input * input;
    if (input < 0) {
        output = output * -1;
    }
    return output;
}
```

`double`	the **return type** — what comes out
`squareInputWithSign`	the **name** — camelCase
`(double input)`	the **parameters** — what goes in
`{ ... }`	the **body** — the actual work
`return`	hands the answer back and stops

If a method hands nothing back, its return type is `void`. You have already seen two: `init()` and `loop()`.

Read more: Book §5.2 “Class Methods”, Listing 5.2

`public` and `private`: who is allowed in your room?

Keyword	Who can see it	Think of it as
`private`	only this class, nobody else	your diary, in a locked drawer
`protected`	this class, its children, and the same package	your bedroom, plus your own kids
`public`	absolutely everyone	the front garden — anyone can look

> **The rule: be as restrictive as makes sense.** Start with `private`. Open it up only when something outside genuinely needs it. The less you expose, the less can break.

You have already written `public` without noticing: `public void init()`. It is public because the SDK, which is outside your class, has to be able to call it.

`static` means one copy for everybody.

Normally each object gets its own variables. `static` says: **no — there is only ever one of these, shared by all.**

```
// call it without making an object first
double deg = Math.toDegrees(1.57);
```

You never make a `Math` object. You just use it. That is because its methods are `static`.

- **static variable** — usually a constant every object should share
- **static method** — callable without making an object first

A `static` method cannot touch normal (non-static) variables. It does not belong to any one object, so there is no “my” variable for it to reach.

You will meet `static` again in the dashboard config classes, where tunable numbers are shared across the whole program.

Read more: Book §5.5 “static”, §19.1 “Math class”

`final` means: write it once, then it is frozen.

```
private final double GATE_OPEN = 0.3;

private final double GATE_CLOSED = 0.7;
```

Try to change a `final` value later and the program will not even build. Java stops you at your desk instead of on the field.

> **Use it for anything that must not change mid-match:** servo positions, speed limits, field measurements, starting positions.

It is not about being strict for its own sake. It is a note to the next person — often future you at 11pm before a competition — saying **this number is deliberate, do not fiddle with it.**

An @ is a sticky note for the compiler.

Annotation	What it does	What happens if you forget
`@TeleOp()`	puts your program in the TeleOp list on the Driver Station	**it never appears.** You will think your code is broken. It is not.
`@Autonomous()`	same, but in the Autonomous list	same problem
`@Disabled`	still compiles, but hides it from the list	handy for test code you want out of the way at a competition
`@Override`	says “I am deliberately replacing a method I inherited”	it still works, but the compiler stops protecting you from typos

The book calls `@TeleOp()` **CRITICAL**, in capital letters. Missing it is the single most common reason a rookie’s OpMode “does not exist” on the Driver Station.

Read more: Book §1.2.4 (annotations), Chapter 14 (@Override)

JAVA — RECAP

Sixteen words you now own.

`int`	whole number
`double`	number with decimals
`boolean`	true or false
`String`	text
`if / else`	make a decision
`while / for`	repeat something
`switch`	pick one of many options
`enum`	a short list of allowed names

`return`	hand an answer back
`void`	hands nothing back
`public`	everyone may use it
`private`	only this class may use it
`static`	one copy shared by all
`final`	set once, never changes
`@TeleOp`	show up on the Driver Station
`@Override`	deliberately replacing a parent method

Read more: Book Chapters 2, 4, 5, 12, 19 · gm0 → Software Glossary

2

Object Oriented Concepts

A way of organising code so that it is shaped like the real thing it controls. Instead of one enormous list of instructions, you build separate parts that each look after themselves.

IN THIS PART

1. Everything around you is an object
2. Class and object
3. Abstraction
4. Encapsulation
5. Inheritance
6. is-a versus has-a
7. Back to our robot

Ten words you are about to meet.

object	one actual thing, with its own details
class	the blueprint that thing was made from
field	something an object *knows*
method	something an object *can do*
abstraction	simple on the outside, messy on the inside
encapsulation	an object guards its own details
inheritance	a child class gets its parent’s stuff free
is-a / has-a	the test for which one to use
new	the word that builds an object
extends	the word that sets up inheritance

Read more: Book Chapters 5, 6, 14

Everything around you is already an object.

Look at a dog.

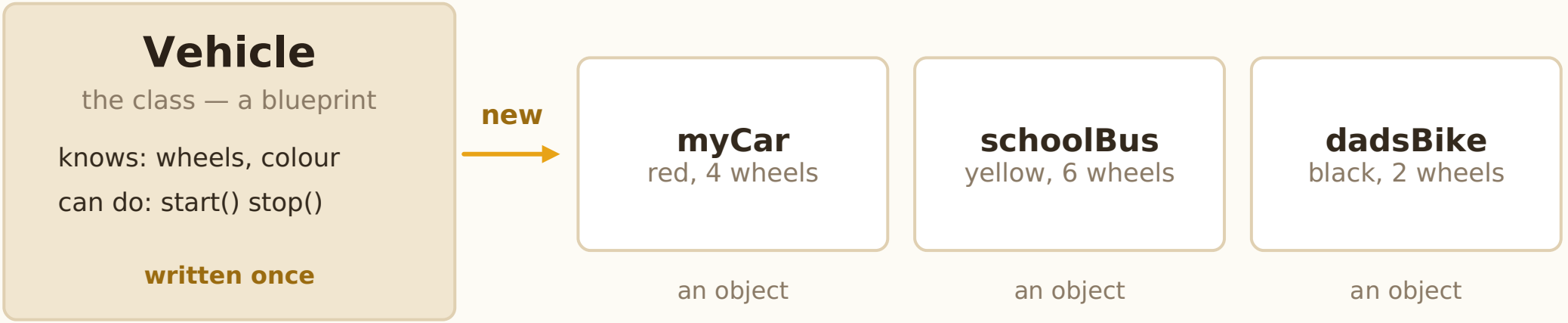
It knows things	its name, its colour, how hungry it is
It can do things	bark, sit, run, eat
It keeps its own state	your dog being hungry says nothing about mine

Now look at a car.

It knows things	its colour, its speed, how much fuel is left
It can do things	start, accelerate, brake, indicate
It keeps its own state	your car’s fuel is not my car’s fuel

That is the whole idea. An object is a thing that **knows** some things and **can do** some things, and looks after its own details. Object oriented programming is just building software out of parts like that.

A class is the blueprint. An object is the actual thing.



one blueprint, as many real vehicles as you like

Vehicle	the class — a description, written once
myCar	an object — a real one, with real details

A blueprint for a house is not a house. You cannot live in it. But you can build a hundred houses from it, and each one gets its own furniture. **That is class versus object.**

Read more: Book Chapter 5, §5.4 “Creating your own classes”

Abstraction: you drive a car without opening the bonnet.

To drive, you need to know four things: steering wheel, accelerator, brake, gear. That is it.

You do **not** need to know about fuel injection, valve timing, or how the engine computer decides anything.

> **Abstraction** is deciding what to show and what to hide. Show a switch, not a wiring diagram.

Real thing	What you actually use
a car	wheel, pedals, gear stick
a television	power, volume, channel
a washing machine	one dial
our Shooter class	`spinUp()` `fire()` `stop()`

> A good class is like a good dashboard: a small number of obvious controls, and all the complicated machinery kept out of sight.

Read more: Book §6.2 “Mechanisms”, Chapter 7

Encapsulation: the bonnet is bolted shut for a reason.

public — what the driver gets

steering wheel

pedals

fuel gauge

ignition key

private — sealed under the bonnet

engine

fuel injection

wiring loom

engine computer

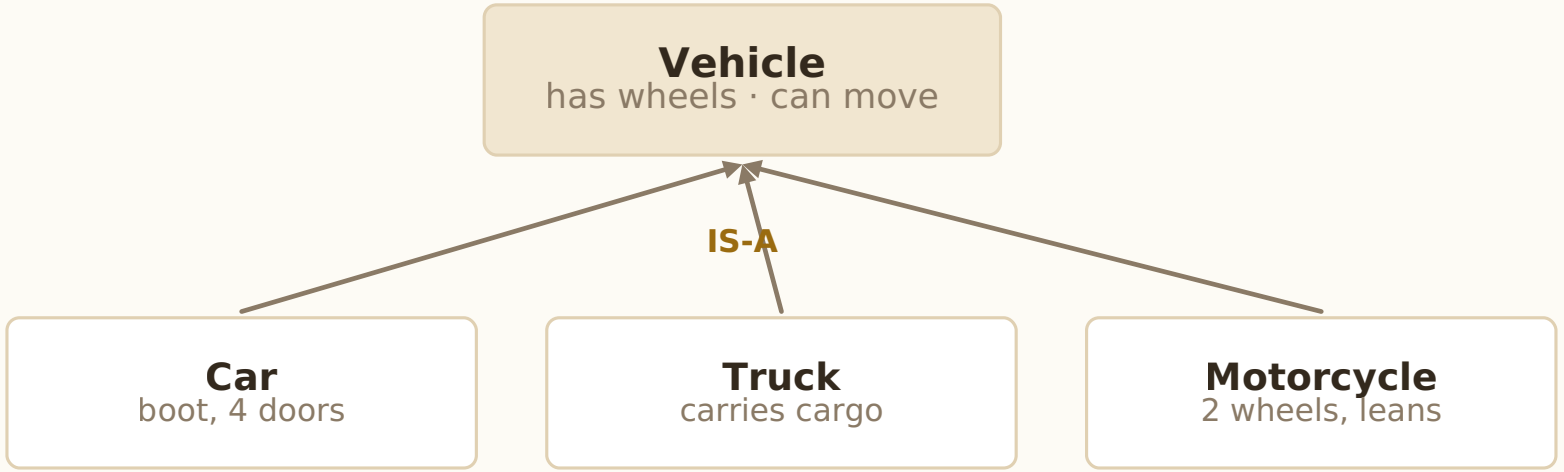
You can drive perfectly well without ever opening the bonnet.
And the maker can change the engine without changing how you drive.

Why bolt it shut? If every driver could reach in and adjust the engine, no two cars would work the same way, and the maker could never change anything without breaking somebody.

In Java this is exactly the `private` and `public` from Part 1, used on purpose. `private` is the bonnet. `public` is the dashboard.

Read more: Book §5.3, §6.2 “Mechanisms”, Listing 6.1

Inheritance: a child class gets its parent’s stuff for free.



Every Car is a Vehicle. Not every Vehicle is a Car.

A Car does not need to re-explain what wheels are, or how moving works. It gets all of that from Vehicle, then adds a boot and four doors.

The animal kingdom works the same way. **Animal → Mammal → Dog**. Each level adds something and keeps everything above it.

Read more: Book Chapter 14 “Inheritance”

Say the sentence out loud. Whichever sounds right, wins.

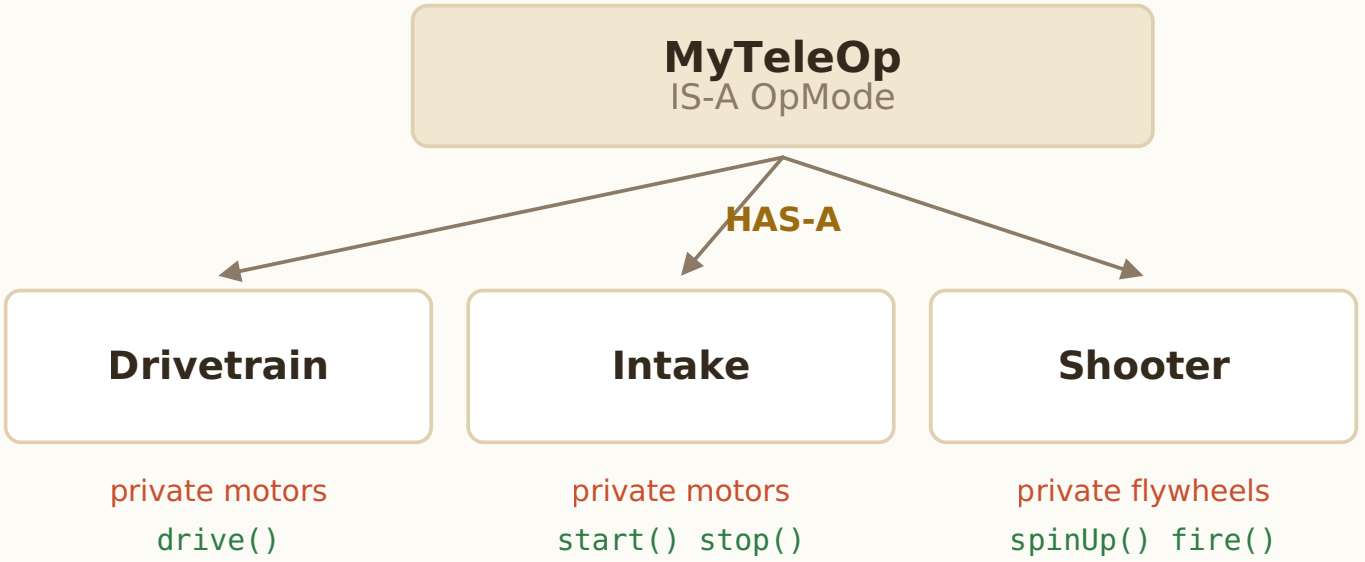
Sentence	Sounds	So you write	Called
“A Car **is a** Vehicle”	right	`class Car extends Vehicle`	inheritance
“A Car **has a** Vehicle”	wrong	—	—
“A Car **has an** Engine”	right	`Engine engine = new Engine();`	a class member
“A Car **is an** Engine”	wrong	—	—

The book’s advice, and it is good advice: start by making it a class member (has-a). Only use `extends` when it is really, obviously what you need. Beginners reach for inheritance far too early.

Try it on the robot: “our TeleOp program **is a** OpMode” sounds right. “our robot **has an** intake” sounds right. Both sentences are true, and each one tells you which tool to use.

Read more: Book §14.1 “Isa vs. hasa”

Now the same five ideas, on real robot code.



red = hidden inside green = what others may call

Two reasons this is worth the effort.
One: a bug in the shooter gets fixed in one file, not scattered across the program.
Two: the exact same `Shooter` class works in Autonomous and in TeleOp.

class	`Shooter`, `Intake`, `Drivetrain`
abstraction	`fire()` hides four motor calls
encapsulation	`private DcMotor flywheel;`
inheritance	`MyTeleOp extends OpMode`
has-a	the OpMode has a Shooter

Read more: Book §6.2 “Mechanisms”, §14.1

Ten words you now own.

object	one actual thing, with its own details
class	the blueprint that thing was made from
field	something an object *knows*
method	something an object *can do*
abstraction	simple on the outside, messy on the inside
encapsulation	an object guards its own details
inheritance	a child class gets its parent’s stuff free
is-a / has-a	the test for which one to use
new	the word that builds an object
extends	the word that sets up inheritance

3

Thinking First

Flowcharts, events and state machines. The part that separates code that works from code that works *reliably*.

IN THIS PART

1 Draw it before you type it
2 A real flowchart
3 Events
4 Edge versus level
5 What state means
6 The traffic light
7 enum and switch, together
8 Our shooter
9 The golden rule

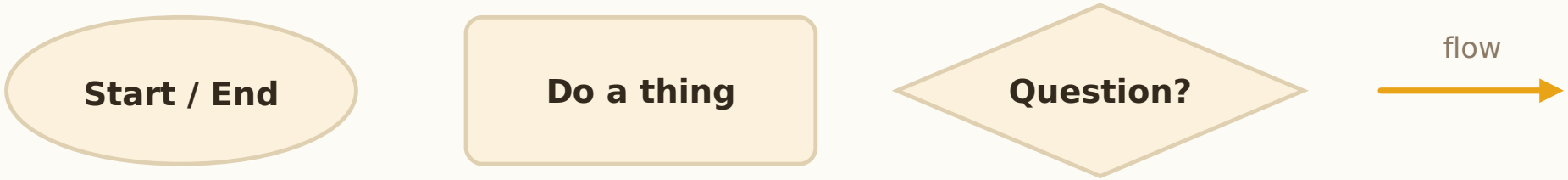
THINKING FIRST — WHAT IS COMING

Ten words you are about to meet.

flowchart	a drawing of what happens in what order
decision	a question with two ways out
event	something happened that we should react to
polling	asking “has it happened yet?” over and over
edge	the moment it changed, not the whole time it is true
state	remembering what you already did
state machine	always in exactly one state
transition	the arrow from one state to the next
trigger	the reason an arrow gets taken
blocking	waiting, and freezing everything else

Read more: Book Chapter 12 · gm0 → Finite State Machines

Draw it before you type it. Paper is cheaper than debugging.



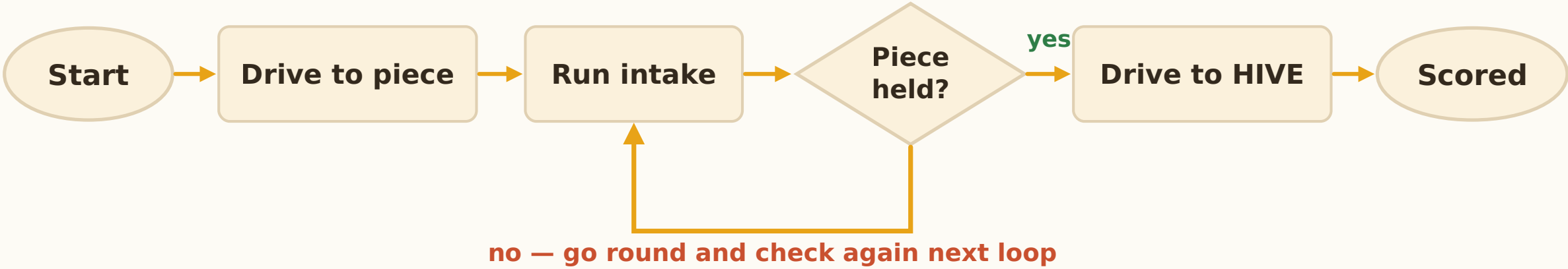
Four shapes. That is the entire language.

> Try it on something you already know: waking up, checking the time, deciding whether to rush or take it slow, getting the bag, leaving. Every program you will ever write has that same shape.

Read more: gm0 → Finite State Machines

THINKING FIRST — A REAL ONE

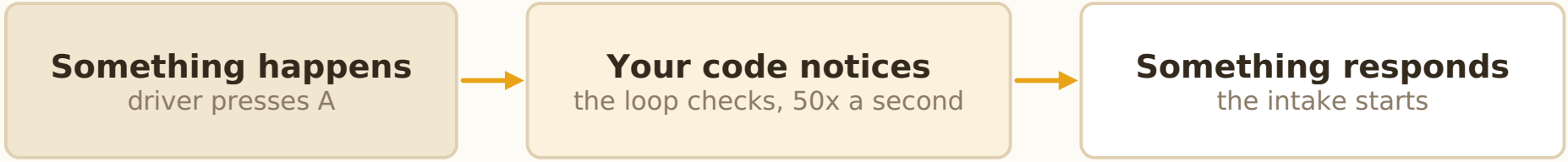
Same four shapes. Real job: score one game piece.



Notice the loop on the left. If the piece is not held yet, we do not freeze and wait — we go round again and check next time. **That distinction is the whole rest of this part.**

Read more: Book Chapter 12 “Dealing with State”

An event is just: something happened, and we care.



that whole chain is one EVENT

In FTC nothing taps you on the shoulder. Your loop has to keep asking.

Events are everywhere. A doorbell. A phone buzzing. A kettle clicking off. Something changes, and something else reacts to it.

In FTC there is no doorbell. Nothing interrupts your program to tell it a button was pressed. Your `loop()` has to keep asking — fifty times a second. That asking is called **polling**.

Read more: Book Chapter 3 (gamepad), Chapter 12 “Dealing with State”

Holding a button down is not one event. It is fifty.

Button held down for one second:

true	true	true	true	true	true	true	true	true	true

Level → 50 events a second. Your intake toggles 50 times.

NEW	same	same	same	same	same	same	same	same	same

Edge → one event. That is what you almost always want.

This is the bug every rookie writes once. “Press A to open the gate” becomes “the gate flaps open and shut fifty times a second for as long as you hold A.” The fix is to react to the **change**, not to the state — and to do that, you have to remember what the button was doing last time.

State is remembering what you already did.

```
boolean aAlreadyPressed;
boolean motorOn;

@Override
public void loop() {
    if (gamepad1.a && !aAlreadyPressed) {
        motorOn = !motorOn;
    }
    aAlreadyPressed = gamepad1.a;
}
```

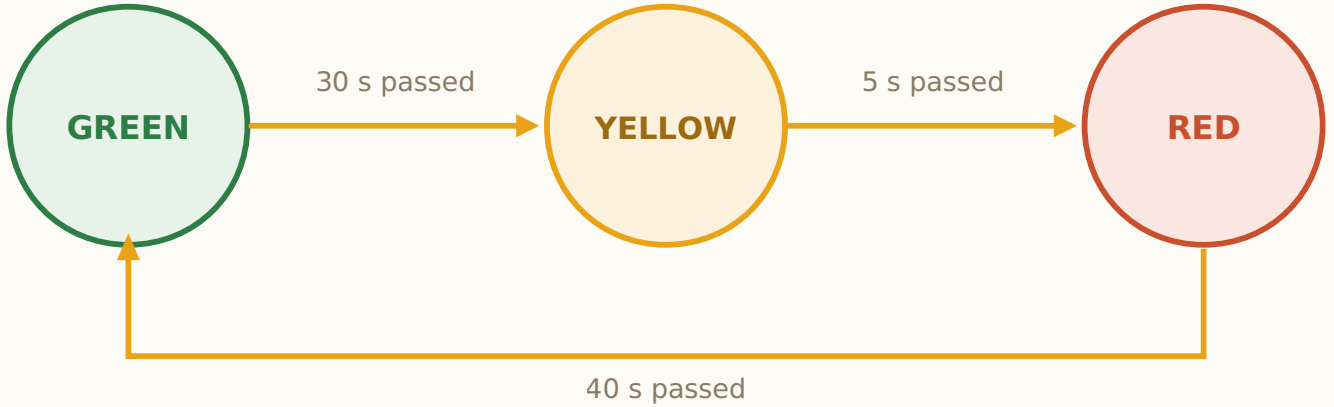
Press A once: motor on. Press A again: motor off. Simple to describe — but impossible without memory.

That last line is the whole trick. `aAlreadyPressed` remembers what the button was doing last loop, so the code can spot the **moment it changed**.

Both variables sit at **class level**, not inside `loop()`. Remember slide 10 — this is exactly why that mattered.

Read more: Book §12.1 “A simple example”, Listing 12.1

A traffic light is a state machine. So is your shooter.



One state at a time. Every arrow needs a reason.

- The light is in **exactly one** state. Never two.
- Arrows are **transitions**. Each needs a trigger.
- The light does not “wait”. It checks the clock and moves on.
- Draw the circles and arrows **before** writing any Java.

Read more: gm0 → Finite State Machines

enum and switch, doing a real job together.

```
enum State {
    START,
    WAIT_FOR_SENSOR_RELEASE,
    WAIT_FOR_POT_TURN,
    STOP,
    DONE
}

State state = State.START;

@Override
public void loop() {
    switch (state) {
        case START:
            board.setServoPosition(0.5);
            if (board.isTouchSensorPressed()) {
                state = State.WAIT_FOR_SENSOR_RELEASE;
            }
            break;
        ...
    }
}
```

You already know both halves. **The enum holds the circles. The switch holds the arrows.**

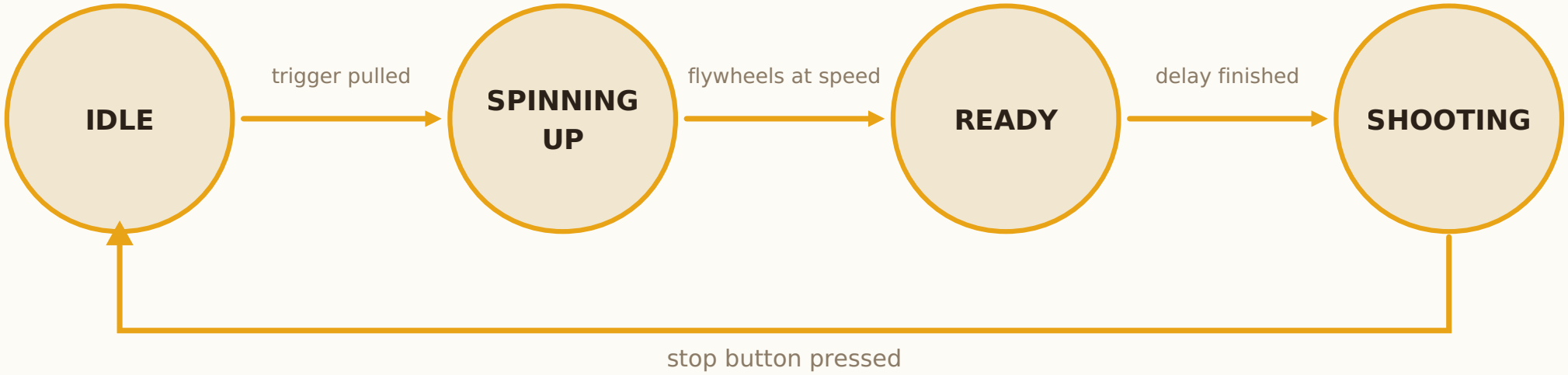
`enum State`	the list of circles on your drawing
`State state`	which circle we are in right now
`case START:`	what to do while in that circle
`state = ...`	take an arrow to the next circle

> Each `case` does two things only: **do the work for this state**, then **check whether it is time to move on**.

Read more: Book §12.2.1, §12.2.3 “Enumerated types”, Listing 12.5

THINKING FIRST — OURS

Our shooter, drawn the same way.



Read it aloud: **idle until the trigger is pulled; spin up until the flywheels reach speed; wait for the delay; then fire until told to stop.** Four circles and four arrows — a complete, testable design, before a single line of Java exists.

THINKING FIRST — THE GOLDEN RULE

Never make the loop wait.

DO NOT DO THIS

```
// robot is now frozen
while (!sensorPressed()) {
}
setMotorSpeed(0.0);
```

While that loop spins, **nothing else on the robot runs.** The drive stops responding. Telemetry stops. Events are missed.

DO THIS

```
// check, act, and get out
case WAIT_FOR_SENSOR:
    if (sensorPressed()) {
        setMotorSpeed(0.0);
        state = State.DONE;
    }
    break;
```

Ask the question, do something if the answer is yes, and **return immediately**. The loop will come back in twenty milliseconds and ask again.

Read more: Book Appendix B.2.2 · gm0 → Finite State Machines

THINKING FIRST — RECAP

Ten words you now own.

flowchart	a drawing of what happens in what order
decision	a question with two ways out
event	something happened that we should react to
polling	asking “has it happened yet?” over and over
edge	the moment it changed, not the whole time it is true
state	remembering what you already did
state machine	always in exactly one state
transition	the arrow from one state to the next
trigger	the reason an arrow gets taken
blocking	waiting, and freezing everything else

Read more: Book Chapter 12 · gm0 → Finite State Machines

4

The FTC SDK

The pile of code FIRST already wrote, so that your Java can reach real motors. Check which version your project is on — it changes each year.

IN THIS PART

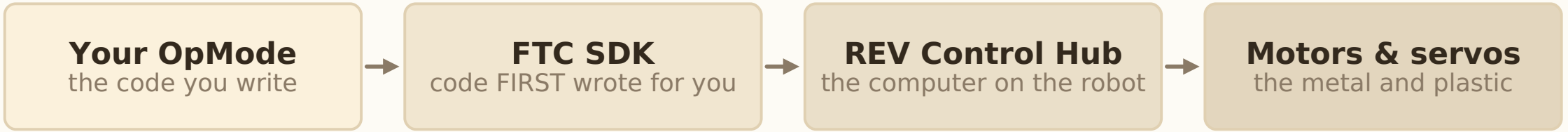
1 The SDK is a translator
2 The OpMode lifecycle
3 init() resets everything
4 hardwareMap and names
5 Motors and servos
6 The gamepad
7 Telemetry
8 Dashboards

Eleven words you are about to meet.

SDK	code FIRST wrote so you can reach hardware
OpMode	one program on the robot
`init()`	runs once when INIT is pressed
`loop()`	runs ~50 times a second after PLAY
`hardwareMap`	how code finds a real motor by name
configuration	the list of devices on the Driver Station
`setPower()`	how hard a motor turns, -1.0 to 1.0
`setPosition()`	where a servo should go, 0.0 to 1.0
gamepad	the controller, read fresh every loop
telemetry	how the robot talks back to you
dashboard	a web page for watching and tuning live

Read more: Book Chapters 1, 6, 7, 8 · gm0 → Software

The SDK is your translator.



each layer only talks to the one next to it

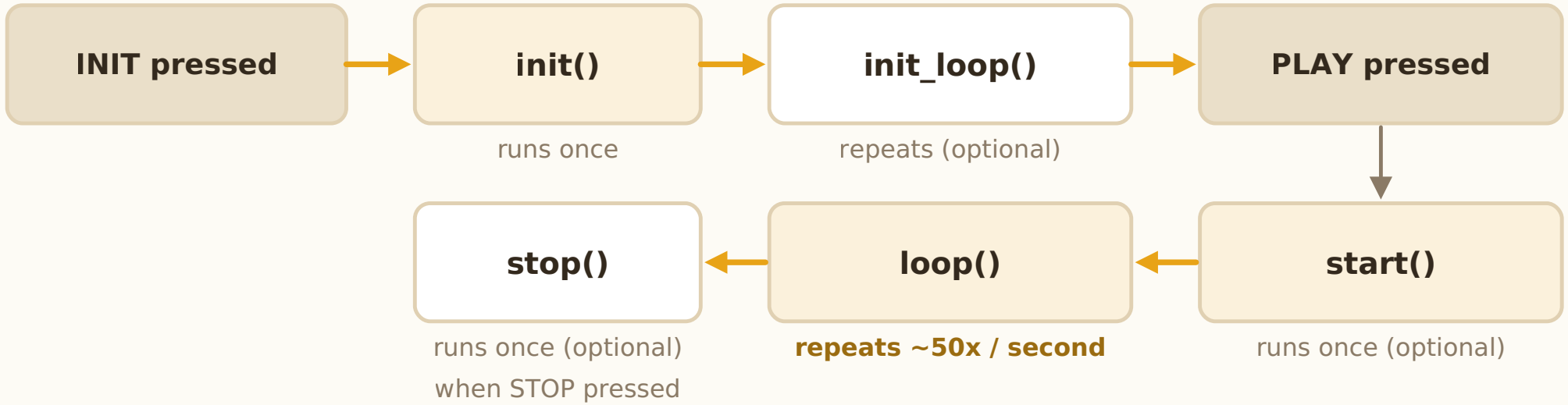
SDK stands for Software Development Kit. It is a large pile of Java that someone else wrote and tested, so that you never have to speak to a motor controller directly.

> You write `motor.setPower(0.5)`. Underneath, the SDK turns that into electrical signals on a cable. You do not need to know how. That is abstraction, from Part 2, at industrial scale.

> It also explains all those `import` lines at the top of every file. Each one says: **fetch me this bit of the SDK.**

Read more: Book §1.1 “Hardware”, Chapter 18 “Finding things in FTC SDK”

An OpMode is one program. `init` and `loop` are its heartbeat.



- **Required:** `init()` and `loop()`
- **Optional:** `init_loop()`, `start()`, `stop()`

> You can have as many OpModes as you like. Each one is a separate class in its own file, and each one appears by name on the Driver Station.

Read more: Book §1.2.3 “Parts of an OpMode”

`init()` puts everything back to normal.

```java
@Override
public void init() {
    robotLocation.setAngle(0);
}
```

Imagine: you run the OpMode, stop it halfway, then press INIT and run it again. Without a reset, the robot still remembers where it got to last time.

The book states it as a best practice, and we will follow it: your `init()` method should set things back to their expected default state. Gate closed. Counters at zero. Arm down.

You will be very glad of this at a competition, when you are resetting between matches under time pressure.

Read more: Book §5.4 (best practice note)

hardwareMap is the robot's contact list.

```
DcMotor leftFront = hardwareMap.get(
        DcMotor.class, "left_front");
```

That string must match the configuration file on the Driver Station, character for character. Capital letters count. Underscores count. Spaces count.

THE THREE-STEP ROOKIE CHECKLIST

- Is the device **in** the configuration file?
- Does the name match your code **exactly**?
- Is that configuration **activated**? (top right of the screen)

If the answer to any of those is no, your program crashes the moment you press INIT, with a message naming the device it could not find. **Read that message.** It is telling you the answer.

Motors take power. Servos take position.

	Method	Range	Means
Motor	`setPower(x)`	`-1.0` to `1.0`	how hard, and which way. `0.0` is stop, `-1.0` is full reverse.
Servo	`setPosition(x)`	`0.0` to `1.0`	go to this spot and hold there. Not a speed — a destination.

```
motor.setPower(0.5);      // half speed forward
servo.setPosition(0.3);   // move to 30% and hold
```

Motors can also be read, not just written — the built-in encoder tells you how far it has turned and how fast it is going. That is how our shooter knows its flywheels are up to speed.

Buttons are true or false. Sticks are decimals.

Part	Type	Range	Example
buttons, bumpers, d-pad	`boolean`	`true` / `false`	`gamepad1.a` `gamepad2.dpad_up`
joysticks	`double`	`-1.0` to `1.0`	`gamepad1.left_stick_x`
triggers	`double`	`0.0` to `1.0`	`gamepad2.right_trigger`

The one that catches everybody: on a joystick, **pushing up gives you a negative number.** That is why almost every FTC program you will ever read has a minus sign in front of `left_stick_y`.

There are two gamepads: `gamepad1` and `gamepad2`. Most teams put driving on gamepad 1 and mechanisms on gamepad 2. Both are re-read fresh every loop — that is the polling from slide 35.

Telemetry is how the robot talks back to you.

```
telemetry.addData("Motor Speed", motorSpeed);
telemetry.addData("State", state);
telemetry.addData("Piece held?", hasPiece);
```

In an `OpMode`, the SDK sends your telemetry for you at the end of every `loop()`. You do not have to ask.

This is your **only** window into a running robot. There is no screen, no keyboard, no way to pause and poke around.

Print your state. When a state machine misbehaves, seeing `State: SPINNING_UP` frozen on the Driver Station tells you instantly which transition never fired. Guessing tells you nothing.

A dashboard lets you watch numbers and turn knobs, live.

Telemetry on the Driver Station is text only. A dashboard is a web page on your laptop showing graphs — and letting you change values **without rebuilding the app**. That turns an hour of tuning into ten minutes.

	FTC Dashboard	Panels
What it is	The long-established option many FTC teams use	A newer one, shipped with the Pedro Pathing quickstart
Good for	live graphs, and values you can tune from the browser	the same job, with Pedro integration built in

Pick one and standardise on it. Set both up on a laptop, try them on the real robot, then choose. Mixing the two across a codebase causes more confusion than either one saves.

Read more: Pedro Pathing Quickstart (Panels integration)

Eleven words you now own.

SDK	code FIRST wrote so you can reach hardware
OpMode	one program on the robot
`init()`	runs once when INIT is pressed
`loop()`	runs ~50 times a second after PLAY
`hardwareMap`	how code finds a real motor by name
configuration	the list of devices on the Driver Station
`setPower()`	how hard a motor turns, -1.0 to 1.0
`setPosition()`	where a servo should go, 0.0 to 1.0
gamepad	the controller, read fresh every loop
telemetry	how the robot talks back to you
dashboard	a web page for watching and tuning live

Read more: Book Chapters 1, 6, 7, 8 · gm0 → Software Glossary

5

Closing the Loop

Back to where we started.

IN THIS PART

1. Read the slide-4 program
2. Cheat sheet
3. What happens next
4. Everything we cited

Remember slide 4? Read it now.

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.OpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;

@TeleOp()
public class HelloWorld extends OpMode {
    @Override
    public void init() {
        telemetry.addData("Hello", "World");
    }

    @Override
    public void loop() {
    }
}
```

`package`	which folder this file lives in
`import`	fetch these pieces of the SDK
`@TeleOp()`	show up on the Driver Station
`public class`	everyone may use this recipe
`extends OpMode`	inherit telemetry, gamepads, hardwareMap
`@Override`	deliberately replacing a parent method
`void init()`	runs once, hands nothing back
`void loop()`	runs ~50 times a second

You just read a real program.

Read more: Book §1.2.4, Listing 1.1

CHEAT SHEET

One sentence from each part. Photograph this slide.

Part	The one thing to remember
0 — The loop	Your robot reads, decides and acts about fifty times a second. Everything else is detail.
1 — Java	Say the type, say the name, end with a semicolon. Braces are walls. One class, one file.
2 — Objects	An object knows things and does things. Hide what you can; expose only what others need.
3 — Thinking first	React to the change, not the state. One state at a time. Never make the loop wait.
4 — FTC SDK	Names in `hardwareMap` must match the config exactly. Print your state to telemetry.

Read more: Every slide in this deck cites its source at the bottom. Nothing here is invented.

WHAT HAPPENS NEXT

Three jobs this week. All of them small.

1. Read **Chapter 1** of Learn Java for FTC and type out `HelloWorld.java` yourself. Do not copy-paste. Typing it is the point.

2. Make it say **your own name** instead, and add your grade as an `int`. (That is exercise §2.4 in the book.)

3. On paper, draw the **state machine** for one thing our robot does. Circles and arrows. No Java.

Getting stuck is the job, not a sign you are bad at this. Every person who writes robot code spends most of their time not knowing why something is broken. The skill being built is finding out — not already knowing.

Before you ask anyone for help: what did you expect, what actually happened, and what does telemetry say? Having those three answers ready solves about half of all problems on its own.

Read more: Book Chapter 1, §2.4 Exercises · Sample solutions: github.com/alan412/LearnJavaForFTC

EVERYTHING CITED IN THIS DECK

Three sources. Bookmark all of them.

Source	What it is	Where
Learn Java for FTC	The book this deck follows. Free PDF, with exercises.	`github.com/alan412/LearnJavaForFTC`
Game Manual 0	Community guide to everything FTC.	`gm0.org`
FIRST game & season	The official game manual. Updated during the season.	`firstinspires.org/programs/ftc/game-and-season`

One rule about anything else you find online: check the date and the version. FTC changes every year. An answer that was right in 2024 may be wrong today.

Pedro Pathing — how the robot drives itself across the field — is covered in a separate deck.